The Edward S. Rogers Sr. Department of
Electrical and Computer Engineering
University of Toronto

ECE496Y Design Project Course
Group Final Report

Title: 400G Ethernet Exploration — An FPGA-Accelerated Platform for BER Analysis Across Modulation, Receiver, and FEC Architectures

Team #:	2025148	
Team members:	**Name:**	**Email:**
	Kevin Jiao	kevin.jiao@mail.utoronto.ca
	Suet Ling (Celine) Kwan	celine.kwan@mail.utoronto.ca
	Elizabeth Tyra Sumual	tyra.sumual@mail.utoronto.ca
	Jiaxin (Brett) Yang	brett.yang@mail.utoronto.ca
Supervisor(s):	Prof. Tony Chan Carusone	
Section #:	7	
Administrator:	Mr. Matthew Yee	
Submission Date:	March 21, 2026	

Group Final Report Attribution Table

This table should be filled out to accurately reflect who contributed to each section of the report and what they contributed. Provide a **column** for each student, a **row** for each major section of the report, and the appropriate codes (e.g. 'RD, MR') in each of the necessary **cells** in the table. You may expand the table, inserting rows as needed, but you should not require more than two pages. The original completed and signed form must be included in the hardcopies of the final report. Please make a copy of it for your own reference.

Section	**Student Names**			
	K. Jiao	S. L. Kwan	E. T. Sumual	J. Yang
I. Group Highlights	MR			RS, RD
II. Individual Contribution	RD	RD	RD	RD
III. Acknowledgement	RS, RD		MR	
IV. Executive Summary			RS, RD	MR
1. Introduction	RS, RD	MR		
2.1. System Block Diagram	MR		RS, RD	
2.2.1. AWS Overview		MR		RS, RD
2.2.2. RTL Overview		RS, RD	MR	
2.3.1. AWS Module-level Description	MR			RS, RD
2.3.2. RTL Module-level Description		RS, RD		MR
2.4. Assessment of Final Design	MR	MR	MR	RS, RD
3.1 Verification Table	MR			RS, RD
3.2.1 Feasibility Testing			RS, RD	MR
3.2.2. Hardware Model Verification	MR		RS, RD	
3.2.3. Software Reference Model	MR	RS, RD		
3.2.4 AWS Setup Verification			MR	RS, RD
3.3 Final Test Results	MR		RS, RD	MR
4.0 Summary and Conclusions	RS, RD	MR		
Appendix A		RS, RD		
Appendix B				RS, RD
Appendix C	RS, RD	RS, RD	RS, RD	RS, RD
Appendix D			RS, RD	

Appendix E		RS, RD		
All	ET, FP	ET, FP	ET, FP	ET, FP

Abbreviation Codes:

RS – responsible for research of information

RD – wrote the first draft

MR – responsible for major revision

ET – edited for grammar, spelling, and expression

OR – other

"All" row abbreviations:

FP – final read through of complete document for flow and consistency

CM – responsible for compiling the elements into the complete document

OR - other

Signatures

By signing below, you verify that you have read the attribution table and agree that it accurately reflects your contribution to this document.

Name	Kevin Jiao	**Signature**	Kevin Jiao	**Date:**	Mar. 21, 2026
Name	Suet Ling (Celine) Kwan	**Signature**	Suet Ling Kwan	**Date:**	Mar. 21, 2026
Name	Elizabeth Tyra Sumual	**Signature**	Elizabeth Tyra Sumual	**Date:**	Mar. 21, 2026
Name	Jiaxin (Brett) Yang	**Signature**	Jiaxin Yang	**Date:**	Mar. 21, 2026

Voluntary Document Release Consent Form

To all ECE496 students:

To better help future students, we would like to provide examples that are drawn from excerpts of past student reports. The examples will be used to illustrate general communication principles as well as how the document guidelines can be applied to a variety of categories of design projects (e.g. electronics, computer, software, networking, research).

Any material chosen for the examples will be altered so that all names are removed. In addition, where possible, much of the technical details will also be removed so that the structure or presentation style are highlighted rather than the original technical content. These examples will be made available to students on the course website, and in general may be accessible by the public. The original reports will not be released but will be accessible only to the course instructors and administrative staff.

Participation is completely voluntary and students may refuse to participate or may withdraw their permission at any time. Reports will only be used with the signed consent of all team members. Participating will have no influence on the grading of your work and there is no penalty for not taking part.

If your group agrees to take part, please have all members sign the bottom of this form.

Consent Statement

We verify that we have read the above letter and are giving permission for the ECE496 course coordinator to use our reports as outlined above.

Team #: 2025148

Project Title: 400G+ Ethernet Exploration — An FPGA-Accelerated Platform for BER Analysis Across Modulation, Receiver, and FEC Architectures

Supervisor(s): Prof. Tony Chan Carusone **Administrator:** Mr. Matthew Yee

Name	Kevin Jiao	**Signature**	Kevin Jiao	**Date:**	Mar. 21, 2026
Name	Suet Ling (Celine) Kwan	**Signature**	Suet Ling Kwan	**Date:**	Mar. 21, 2026
Name	Elizabeth Tyra Sumual	**Signature**	Elizabeth Tyra Sumual	**Date:**	Mar. 21, 2026
Name	Jiaxin (Brett) Yang	**Signature**	Jiaxin Yang	**Date:**	Mar. 21, 2026

Group Highlights (author: J. Yang)

This section summarizes the key achievements and final outcomes of the Capstone project, with emphasis on the final results.

The key achievements include:

1. **Cloud-Based FPGA Accelerator for Wireline Channel Simulation:** Developed and successfully deployed a fast and scalable channel modeling platform on Amazon Web Services (AWS) tailored for the evaluation of future 400G Ethernet architectures.
2. **Multi-System Deployment:** With reference to previous work by Richard Barrie targeting 200G PAM-4 architecture, three PAM-6 channel models were built and implemented in the cloud, and they were ready to be used by future researchers.
3. **High-Fidelity RTL Migration & Validation:** Selected system with high complexity maximum-likelihood sequence estimation (MLSE) computations for migration and ensured system correctness via various validation methods.

The key final outcomes include:

1. **Massive Throughput Increase:** A significant performance leap over traditional software methods was achieved. A single-core hardware accelerator provides more than 500x speed-up compared to software baseline models. When scaled to a 100-core cluster, the system offers >50,000x speed improvement, reducing simulation times from years with software models to hours with hardware acceleration.
2. **Cost-Effective Scalability & Portability:** The AWS-based solution eliminates the high upfront costs and maintenance associated with physical FPGA environments. This portable architecture allows researchers worldwide to access high-performance simulation tools with a minimal learning curve via a simplified Python-based graphical control interface.
3. **Verified System Correctness:** Cross-validation was performed between the hardware simulator and its software reference model. The cloud platform was also validated by comparing its bit error rate (BER) to signal-to-noise ratio (SNR) curves with those produced by the reference model of the same system. This ensures that the accelerated results are reliable and accurate.
4. **User-friendly Interface & Documentation:** The platform is delivered with a modular design accompanied by comprehensive user documentation and video tutorial, providing a low barrier to entry for future researchers to replicate results, modify configurations, and conduct further developments.

Individual contribution

This section summarizes the key personal contributions and achievements by each member.

Jiaxin Yang (author: J. Yang)

My primary focus in this project was to (1) bring-up the PAM-6 version of System 6 from the reference design and (2) to implement the infrastructures on the AWS FPGA platform, which includes (i) creating the hardware register-transfer-level (RTL) shell design, (ii) designing and implementing the graphical user interface (GUI) application, and (iii) creating the user documentation and video tutorial.

At the early stage of the project, I successfully expanded the System 6 in the reference design from PAM-4 to PAM-6, which allowed my teammate to test the feasibility of the hardware design on a local FPGA platform. This also provided me with a baseline to be used when I was building the AWS infrastructures at the later stages.

To make it possible for our RTL design to be deployed on the AWS platform, a shell layer was required to act as a translator between our design and AWS. I designed this custom logic wrapper using an AXI4-Lite interface, which utilizes a register-map to allow the software host to control and read simulation statistics out of it. There were problems encountered, and due to no physical access to the hardware platform, I designed debug modules to help me pinpoint the root cause of the dead hardware.

I also designed and implemented the software controller host and its graphical user interface (GUI) on the AWS platform, which is an automated visualization program that abstracts the complexity of the underlying hardware. This program performs automated BER-SNR sweeps and generates live plots of Pre-FEC BER, Post-FEC BER, and frame error rate (FER) waterfall curves. This tool transformed a complex command-line workflow into a user-friendly, visualized experience.

Finally, I focused on making the platform sustainable for future users. I produced comprehensive documentation and a video tutorial that guides future users through the entire process, from AWS machine creation to bitstream generation and to GUI usage. I also actively simplified the process by writing scripts that condensed over 10 manual commands into a single-line execution script. These efforts helped my teammates bring the other two systems to the cloud platform, and more importantly, ensured that future students or researchers could build upon our work with a minimal learning curve, specifically for future goals like PAM-8 expansion or new algorithms for wireline channels.

Suet Ling Kwan (author: S. L. Kwan)

Prior to the interim demo, my main responsibility was to make sure reference model works for System 6 without noise input. I debugged modular components such as the gray coding and MLSE model given a reference PAM-6 implementation from Brett. This included understanding the MLSE function, changing the traceback length, and changing the y-table indexing for the survivor path, and changing the noise input to 0dB noise. This helped make progress on the requirement that we have a software baseline model that verifies the correctness of the RTL model, in this case under no noise condition.

I also assisted with the debugging of core System 6 components – I was able to root cause an issue causing unexpected BER rates in the PAM-6 System 6 model. I spotted an issue related to the misalignment of the gray coding input with the pseudo-random binary sequence (PRBS) input, causing the output in-phase(I) and quadrature(Q) signals to represent the wrong series of bits. This helped achieve the requirement that the system must support BER analysis of PAM-6 modulation under configurable noise conditions.

After the interim demo, my main responsibility and achievements were (1) modifying the reference model such that it works with noise and ensured it matches with our PAM-6 RTL implementation, and (2) adding the final component – the post-FEC module -to the reference model, and (3) building scripts to generate BER-SNR curves to assess the performance of our software simulation model. The first item required debugging the existing noise generation module such that it works with 10dB to 20dB noise – our area of interest to evaluate BER-SNR performance. This required understanding core components within the noise generation data path, including the uniform random noise generator (URNG), random noise, and noise adder modules, and troubleshooting them. I was able to root cause and fix issues related to seed input, step functions, off-by-one cycle misalignment errors within the three modules, resulting in a software reference model that matches the RTL model before FEC. For item two, I created the post-FEC model and tested its accuracy with our PAM-6 RTL and AWS implementation, finalizing the reference model and confirmed its bit-true accuracy. For item three, I conducted performance evaluation on the software simulation and generated BER-SNR curves for software for comparison with Brett's BER-SNR results, thus achieving the requirement that the system must log key performance statistics (pre-FEC BER, post-FEC BER) to files for analysis, allowing our team to compare results across platforms.

Kevin Jiao (author: K. Jiao)

To summarize my main contribution across the entire timeframe of the capstone would be early platform exploration with Azure cloud computing, supporting system migration from PAM-4 to PAM-6 across various systems (System 4 & 5 from Richard Barrie's master thesis [11]), and system bring-up on the AWS platform configurated by Brett.

Earlier stages of the project involved exploration of platforms that can support one of the most important goals of our project, for the system to be future proof and scalable. Microsoft Azure was one of the cloud solutions that I researched and determined to be one of the top solutions as it provided simple deployment, FPGA accelerated instants such as the NP series [1], pre-configurated Vivado system environment, and low upfront cost pay-as-you-go architecture. My work here involved development of base system configurations, code development, verification of feasibility, and demonstration of NP series functionality. Though AWS overrode Azure for development, my research established a reliable baseline for cloud FPGA deployment and provided a fully documented fallback architecture.

Throughout capstone, I analyzed the mathematical derivations of forward error correction (FEC) algorithms, applying this theoretical knowledge directly to streamline our RTL debugging process. I then led the migration of Systems 4 & 5 from PAM-4 to PAM-6. This was a significant technical hurdle because the symbol representation no longer directly mapped: PAM-6 involves five bits of binary data to be extracted between two symbols instead of two bits per singular symbol for PAM-4, introducing significant modification for RTL code segments. To overcome the resulting logic and synchronization challenges, I worked on code development and bitstream verification and adjusted the high-level modules to match PAM-6 modulation. I adjusted symbol encoding, decoding, and cross-compared PAM-6 and PAM-4 simulations to ensure accuracy.

Lastly, I conducted system adaptation to enable the systems from PAM-4 to PAM-6 to be brought up to the AWS platform. I conducted the final integration and system verification by integrating my RTL code into the AWS shell. By matching the cloud system's behavior to our local benchmarks, I ensured a seamless transition to the cloud environment, completing the integration.

Elizabeth Tyra Sumual (author: E. T. Sumual)

My primary role in the early stages of the project was to establish a performance baseline by verifying that hardware acceleration could significantly outperform traditional software simulation. Using a local FPGA board, specifically the Opal Kelly XEM7310 Development Board, I completed the feasibility tests on System 6. This involved migrating Richard Barrie's original PAM-4 RTL modules to PAM-6, accuracy testing using RTL simulations on Vivado, and developing a custom Verilog wrapper to interface with the hardware. To manage the testing environment, I wrote a Python control application to monitor and record pre-FEC and post-FEC results. This phase was successful, achieving a throughput of 40 Mbps, a 769x speedup over the RTL simulation rate of 0.052 Mbps. This achieved the project's core requirement for hardware-level speed, allowing for development on cloud-based architecture.

Leading up to the interim demonstration, my focus shifted to ensuring bit-true accuracy between the Verilog hardware and the Python software reference model. During debugging alongside Celine, I identified a discrepancy in the BER results between the two platforms. By tracing the signal path, I isolated the issue to the inter-symbol interference (ISI) class, which is part of the wireline channel model. I discovered that the difference stemmed from fixed-point truncation inherent in the Verilog code, as FPGAs lack native floating-point hardware. I subsequently updated the Python model to be bit-accurate to the Verilog implementation. Combining our collaborative debugging efforts, this ensured the system could reliably support BER analysis under configurable noise conditions.

In the final phase of the project, I took ownership of the system bring-up for System 4 on the AWS platform. Working with Brett and Kevin, I managed the cloud instance configuration and documented the end-to-end workflow required to deploy Verilog code to AWS F2 instances. I successfully developed the hardware wrapper for System 4, supporting both PAM-4 and PAM-6, and conducted performance trials, including FER-SNR and BER-SNR waterfall curve generation, to verify system stability in a cloud environment. To support our testing phase, I secured AWS credits for the team, which facilitated extensive, cost-free system verification. This fulfilled all project requirements, ensuring that the system could operate at speeds far exceeding the limitations of our local Opal Kelly hardware.

Acknowledgement (author: K. Jiao)

We would like to express our sincere gratitude to everyone who supported us throughout the duration of this capstone project:

First and foremost, we would like to thank our supervisor, Professor Tony Chan Carusone, for his guidance, technical expertise, and for providing us with the opportunity to explore the frontiers of next-generation Ethernet simulation.

We would like to thank Mr. Matthew Yee, our project administrator, whose consistent feedback on our documentation and insightful advice on project management was instrumental in keeping our team on schedule and ensuring the professional quality of our deliveries.

We also received valuable technical guidance from former students in Prof. Carusone's lab. We would like to thank Mr. Richard Barrie for his research, open-source RTL implementations, and advice for our project. His work and insights were vital in helping us select and refine our project goals. We also thank Mr. Qasim Li for the technical discussions and insights that helped to shape our implementation strategies.

We would like to acknowledge our communication instructor for the detailed feedback provided during our first oral presentation and proposal phase, which improved the clarity of our technical communication.

Lastly, we would like to thank Prof. Hamid Timorabadi and the ECE Department for their administrative and technical support throughout the year.

Executive Summary (author: E. T. Sumual)

As global demand for data-intensive applications such as artificial intelligence, 8k streaming, and cloud computing rises, the semiconductor industry is transitioning towards 400 Gigabit Ethernet (400G) and beyond. Ensuring the reliability of these high-speed links requires verifying ultra-low bit error rates (BER) of 10^{-12}. This validation process is impractically slow using traditional software simulations, which can require nearly 500 years to encounter a single bit error at such rates. This project addresses this validation issue by delivering a scalable, hardware-accelerated simulation platform deployed on Amazon Web Services (AWS), a high-performance cloud environment.

This project provides a sustainable and modular framework for both industry and academia to validate transceiver performance before fabrication. By bridging the gap between academic research and industry-grade validation, the platform supports the ongoing development of robust 400G Ethernet standards.

The final deliverable is a modular, scalable, cloud-enabled field programmable gate array (FPGA) framework for simulating a complete 400G wireline transceiver, including:

- Advanced modulation schemes with configurable formats, specifically PAM-4 and PAM-6
- Configurable receiver architectures incorporating maximum-likelihood sequence estimation and forward error correction techniques (e.g. Hamming and Reed-Solomon KP4 codes)
- Cloud-enabled deployment on AWS F2 instances for scalable, parallel FPGA execution, resulting in reduced hardware overhead

The project successfully met all core requirements and performance objectives:

- Significant acceleration with hardware implementation, achieving 570× speed-up on a single core compared to software and scaling to 57,000× improvement across a 100-core cluster
- System correctness validated against a bit-accurate software reference model, providing a baseline of comparison for our hardware and AWS implementations
- User-centric design featuring a Python-based graphical interface for automated parameter sweeps, real-time performance visualization, and simplified interaction with the underlying hardware system

Ultimately, this project streamlines the workflow for wireline communication designers by providing a sustainable, high-speed, and accurate solution for evaluating and optimizing Ethernet performance, while also laying down the groundwork for future development in open-source and open-ended expansions for new architectures and equalization techniques.

Table of Contents

1. Introduction (author: K. Jiao) 1
   1.1 Background 1
   1.2 Motivation 2
   1.3 Project Goals and System Requirements Specification (SyRS) 2
2. Final Design 4
   2.1 System Block Diagram (author: E. T. Sumual) 4
   2.2 System Level Overview 5
      2.2.1 AWS architecture overview (author: J. Yang) 5
      2.2.2 RTL system overview (author: S. L. Kwan) 5
   2.3 Module-level descriptions and designs 7
      2.3.1 AWS Module-level Description (author: J. Yang) 7
      2.3.2 RTL Module-level Description (author: S. L. Kwan) 9
   2.4 Assessment of Final Design (author: J. Yang) 10
      2.4.1 Speedup & Scalability 10
      2.4.2 Correctness 10
      2.4.3 Flexibility 10
      2.4.4 User Friendliness 11
      2.4.5 Trade-offs 11
      2.4.6 Possible Future Improvements 11
      2.4.7 Cost Analysis 12
3. Testing and Verification 13
   3.1 Verification Table (author: J. Yang) 13
   3.2 Detailed Testing Procedures 14
      3.2.1 Feasibility Testing on a local FPGA board (author: E.T. Sumual) 14
      3.2.2 Hardware Model Verification in Vivado (author: E.T. Sumual) 15
      3.2.3 Software Reference Model & Testing (author: S. L. Kwan) 16
      3.2.4 AWS Setup Verification (author: J. Yang) 18
   3.3 Final Test Results (author: E. T. Sumual) 19
      3.3.1 Summary of Requirement Fulfillment 19
      3.3.2 Throughput and Speedup Comparison 19
4.0 Summary and Conclusions (author: K. Jiao) 20
   4.1 Project Summary and Design Validation 20

4.2 Comprehensive Model Verification 20
4.3 Usefulness in Industry and Academia 21
4.4 Future Work and Applications 21
5. References 22
6. Appendices 24
Appendix A. Gantt Chart History (author: S. L. Kwan) 24
Appendix B. Financial Plan (author: J. Yang) 26
Appendix C. Explanation of Technical Background (author: All) 27
Appendix D. More Details on Feasibility Test on Opal Kelly Board (author: E. T. Sumual) 29
Appendix E. System 6 implementation in Python (author: S. L. Kwan) 30

1. Introduction (author: K. Jiao)

This report summarizes the motivation, design, implementation and testing of the "400G Ethernet Exploration — An FPGA-Accelerated Platform for BER Analysis Across Modulation, Receiver, and FEC Architectures" as part of the final year design project course ECE496. The report concludes with suggestions for improvements and future work.

1.1 Background

Ethernet defines how digital data is transmitted across wired networks. The demand for faster and more reliable Ethernet continues to rise, driven by artificial intelligence, cloud computing, and data-intensive applications. As data rates increase toward 400 Gbps and beyond, it becomes much harder to maintain extremely low bit error rates (BER), since signals are more easily distorted by noise and channel imperfections [2] - [5].

To increase the speed of transmission, instead of just transmitting data using the binary encoding of 0s and 1s, engineers started to transmit data using multilevel modulation, where each signal can represent multiple bits by taking on several voltage levels. This has led to the various pulse amplitude modulation (PAM) schemes we see today, such as PAM-4, PAM-6, and PAM-8, where the numbers following PAM indicate the available voltage states. However, higher-level modulation makes signals more sensitive to physical effects such as distortion, timing errors, and interference from neighboring channels [6] – [8]. To maintain reliable communication, designers use forward error correction (FEC), which adds redundant information so that receivers can detect and correct bit errors [9], [10].

Engineers have relied on existing tools to emulate Ethernet wireline characteristics yet have suffered from the tools' inability to model complex systems and their lack of flexibility to test multiple FEC and modulation schemes efficiently [11]. Current tools are mainly implemented in software, whose speed is limited and struggle to evaluate such high-speed systems efficiently, especially when aiming for ultra-low BER levels ($\approx 10^{-12}$) defined by IEEE standards [11].

Despite work in academia proving that hardware acceleration using FPGAs can greatly accelerate the speed of wireline simulations at 200 Gbps speeds [11], there is still a lack of efficient ways to evaluate how well different FEC methods, stronger codes, or layering perform, which is essential for building robust systems for 400G Ethernet [12].

Appendix C includes more detailed explanations of technical terms, like modulation schemes and FEC.

1.2 Motivation

Evaluating higher speed links introduces unique challenges, such as signal integrity issues and exponentially greater simulation complexity. This makes traditional software-based approaches impractically slow. Additionally, platforms to simulate 400G Ethernet FEC architectures standards are not well developed. Our project aims to fill both gaps by expanding Barrie's master's thesis [11] on accelerating wireline simulations utilizing FPGA acceleration to enable evaluation of higher data rates and more complex modulation and FEC architecture that software tools are unable to efficiently evaluate. In addition, we used AWS to ensure future scalability and ease of use. Our platform benefits both industry and academia by helping designers validate transceiver performance, supporting standards development [2], and enabling research into next-generation Ethernet design [2] [13].

In 400G Ethernet, PAM-6 and PAM-8 are strong candidates for modulation [14]. This platform supports PAM-6 testing of different FEC and Receiver (RX) architectures, with PAM-8 expansion becoming an area for future exploration. The resulting framework benefits researchers, semiconductor designers, and Ethernet standard developers by providing a much faster, reusable, modular platform for validating modulation and coding schemes under realistic noise and distortion. Ultimately, it should accelerate innovation and improvement in design confidence for the next-generation 400 G Ethernet and continue to contribute to ongoing Ethernet standard development.

1.3 Project Goals and System Requirements Specification (SyRS)

Our project goal is to build a fast and scalable channel modeling platform on AWS for evaluation of possible 400G Ethernet architectures. Table 1 shows the system requirements specification (SyRS) which shows our requirements, objectives, and constraints.

Table 1: *System requirements specification table*

ID	**Specification**	**Category**
1	The system must support BER analysis of PAM6 modulation under configurable noise conditions.	Requirement
2	The platform must provide modular support for alternative FEC/Receiver architectures.	Requirement

3	The system must have a software baseline model that verifies the correctness of the RTL model	Requirement
4	The system must have faster single-core speed than the Opal Kelly XEM7310 board	Requirement
5	The system must support PAM4 modulation as a baseline to verify correctness against the reference design	Requirement
6	The system must log key performance statistics, including pre-FEC BER, post-FEC BER, and frame error rate (FER), to files for later analysis.	Requirement
7	The system must include a host PC control interface	Requirement
8	The system must support parallel-core execution to enable scalability	Requirement
9	The system must support runtime configuration of channel parameters (e.g., SNR, precoding, interleaving) without resynthesis	Requirement
10	The system must provide sufficient documentation of APIs and configuration steps to enable use by external users	Requirement
11	The system shall be implemented on the AWS FPGA cluster to ensure scalability and reusability for other researchers	Constraint
12	The system shall comply with FPGA resource limitations on available AWS hardware (timing, LUTs, BRAM)	Constraint
13	Implement a software or hardware wrapper for automated sweeping to minimize human effort	Objective
14	Implement pipelining / parallel processing to reduce the timing and achieve higher simulation speed	Objective
15	Design a user-friendly interface (GUI) to enable easier configuration & more visualized results	Objective

2. Final Design

2.1 System Block Diagram (author: E. T. Sumual)

Figure 1 shows the block diagram of our system. It mainly consists of two parts: the AWS platform and the wireline channel model.

The AWS platform is where our wireline channel models are deployed. The register-transfer level (RTL) logic, which is our wireline channel model, is wrapped by the custom logic wrapper so that it is compatible with the platform. It is then synthesized into an Amazon FPGA image (AFI), then uploaded to the AWS F2 Instance. The simulator can then be controlled by the host software, enabling us to start, stop, or obtain results from the instance.

The wireline channel model contains multiple high-level blocks that mimic an Ethernet wireline communication channel, including transmitter, channel, and receiver components. The design is highly modular and developed in a plug-and-play manner. Details of each module are explained in Sections 2.2.2 and 2.3.2.

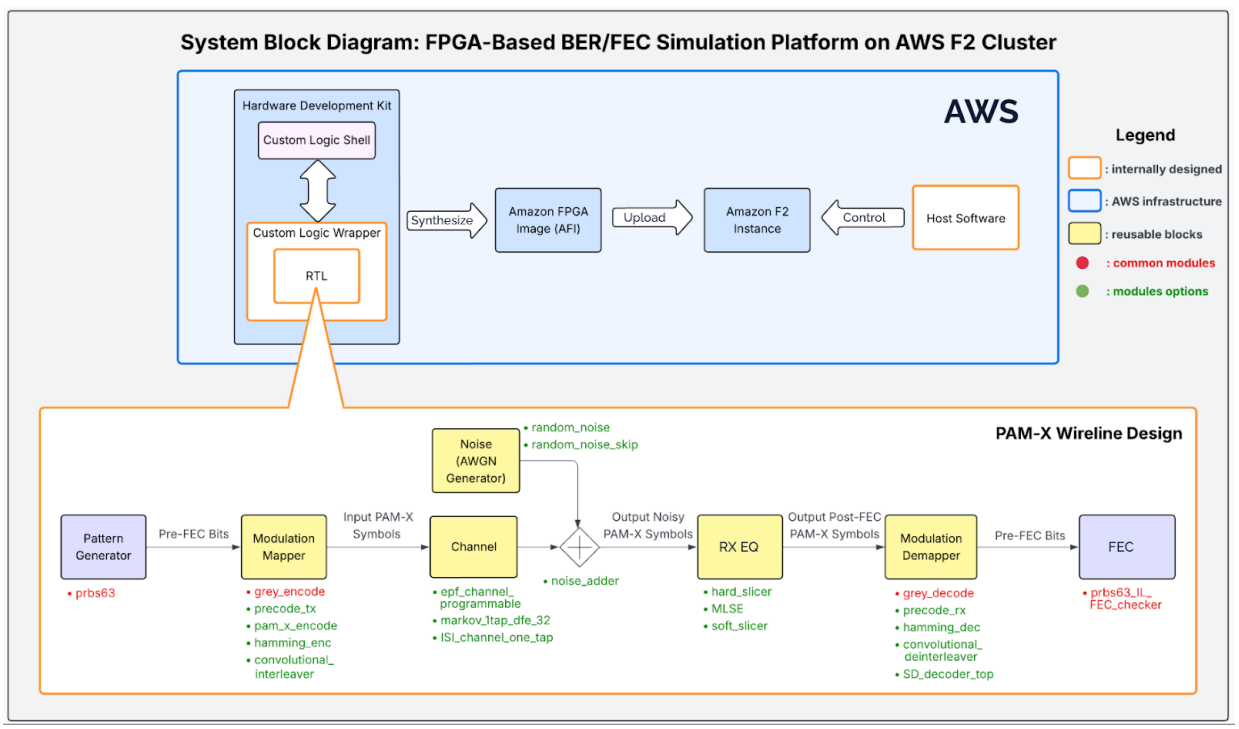


Figure 1: *System block diagram involving AWS (top) and wireline channel (bottom) components.*

2.2 System Level Overview

This section provides a system level overview of the AWS platform and our RTL design.

2.2.1 AWS architecture overview (author: J. Yang)

The simulation platform is hosted on the AWS cloud, more specifically, the AWS EC2 F2 instance. It is a high-performance computer that has a Xilinx FPGA attached [15]. A host software running on the computer acts as a controller of the FPGA design by reading from and writing to the hardware shell interface, which will be elaborated in Section 2.3.1.

To run the design on the cloud, the following steps need to be followed:

1. Connect to the AWS instance and load the Amazon Machine Image (AMI). This serves as the virtualized development workstation, containing the required tools for FPGA development, such as Vivado
2. Implement RTL design, which is our channel model, and wrap it with the custom logic wrapper.
3. Synthesize the design into an Amazon Design Check Point (DCP) and make sure there are no errors or timing violations
4. Generate the Amazon FPGA Image (AFI) from the DCP and deploy it on the FPGA
5. Run the software controller (the GUI)

Section 2.3.1 explains the detailed implementation of the hardware shell and software controller.

2.2.2 RTL system overview (author: S. L. Kwan)

The bottom portion of the system level block diagram shows the hardware implementation of wireline systems. According to Barrie's master's thesis [11], the inputs to the model are driven by pseudo-random binary sequences (PRBS). The usage of PRBS31 is selected as they exhibit statistically random behaviour and are easily generated with linear-feedback shift register (LFSR) circuits, and the Ethernet standard specifies a PRBS31 pattern for testing BER performance. The modulation mapper receives PRBS sequences and applies PAM6 modulation before transmission through the channel. These symbols are represented on the FPGA as three-bit unsigned values (0, 1, 2, 3, 4, 5), which translate to bits in the QAM-32 constellation [11], as shown in Figure 2.

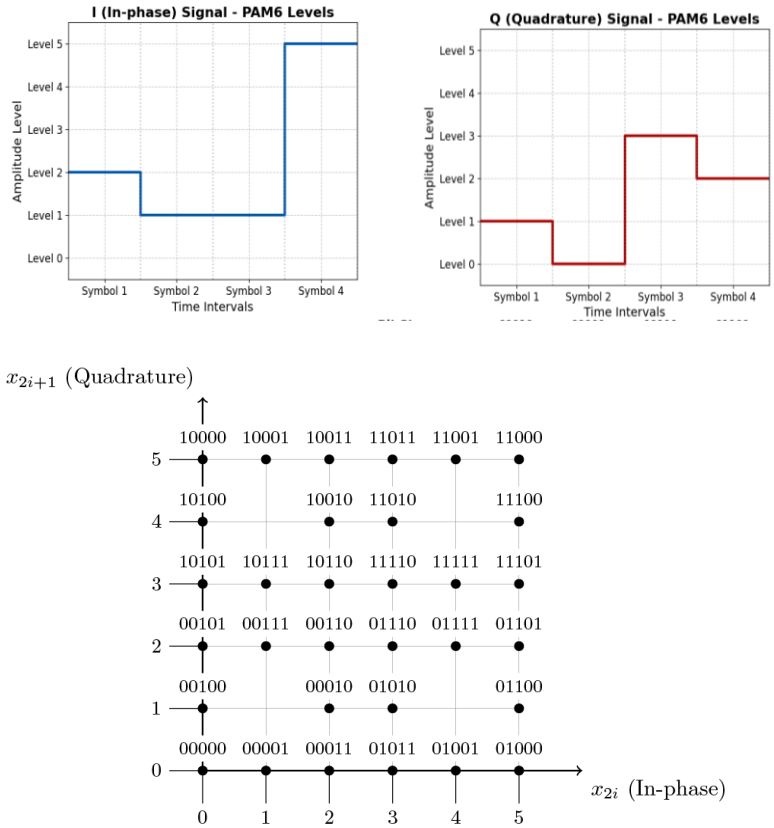


Figure 2: *PAM6 I/Q signals and the QAM-32 constellation [5].*

The channel represents the electrical and optical links as defined in the Ethernet standard. To simulate diverse operating environments, the platform supports modular channel and noise components that can be swapped to represent specific conditions, such as ISI $(1 + \alpha D)$, AWGN, analog LPF coloured noise, to represent different channel conditions. To recover the signal, the received signal passes through the RX equalizer, which employs advanced detection algorithms such as MLSE and the soft-output Viterbi algorithm (SOVA). While MLSE uses the Viterbi algorithm to find the most likely sequence of transmitted symbols, SOVA provides additional reliability information (soft decisions) necessary for high-performance inner-FEC decoding. The equalized data then moves into the modulation mappers, whose purpose is to serialize the symbols back to binary representation so that it can be processed by the final FEC block. This stage utilizes a concatenated architecture - often a combination of a binary Hamming inner code and a Reed-Solomon "KP4" outer code - to correct both random and burst errors. The FEC block also gathers and outputs key performance statistics for BER analysis, such as pre-FEC and post-FEC errors. The specific implementations chosen for this project is detailed in Section 2.3.2.

2.3 Module-level descriptions and designs

This section provides a detailed description of the AWS platform implementation and the RTL implementation.

2.3.1 AWS Module-level Description (author: J. Yang)

Figure 3 illustrates the module-level schematic of the AWS platform custom logic wrapper. The design can be split into three sections based on the clock domains: AXI Domain (blue), Clock Domain Crossing (orange), and Core Domain (green).

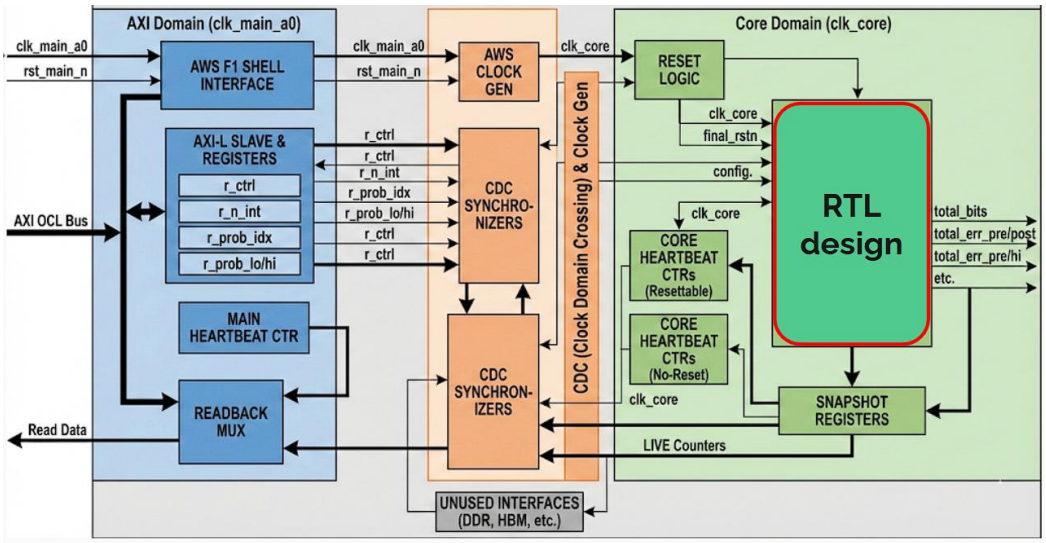


Figure 3: *Module-level schematic of the AWS platform custom logic wrapper.*

The AXI Domain runs on a 250 MHz clock (clk_main_a0) provided by AWS. This frequency cannot be changed otherwise the AXI interface will not work. AXI4-Lite slave registers are implemented to enable read and write commands from the software controller. A main heartbeat counter is introduced for debug purposes. Finally, the readback multiplexer provides the output from the desired slave register for the software host to read.

The Core Domain can be run on any customizable clock frequency, as long as the timing constraints are not violated. It mainly includes the RTL channel simulator (labeled in red), as well as snapshot registers to stabilize the simulator statistics for AXI read. Two heartbeat counters were added for debug purposes.

The Clock Domain Crossing block acts as a bridge between the two clock domains. Synchronizers are needed to allow one domain to communicate with the other without corrupting the data.

The blue block in Figure 4 illustrates the module-level schematic of the AWS platform software host. It is a Python script communicating with the FPGA Hardware using the same AXI4-Lite protocol. The orange block on the right is a simplified version of the hardware wrapper described above.

The software host serves four functions:

1. Control the hardware through AXI address 0x500, which includes design reset and enable
2. Read hardware simulation statistics through AXI address 0x600 - 0x63C. The statistics include pre-FEC error counts, post-FEC error counts, frame error counts, and total bits simulated
3. Load the noise probabilities to the hardware through AXI address 0x508 - 0x514. These parameters are used by the noise generation RTL block to generate noise under a specific level
4. Run the GUI, so the abstract statistics are turned into informative and visualized curves. It also enables automated sweeping with customized sweep parameters. A screenshot of the GUI can be found in Section 2.4.4

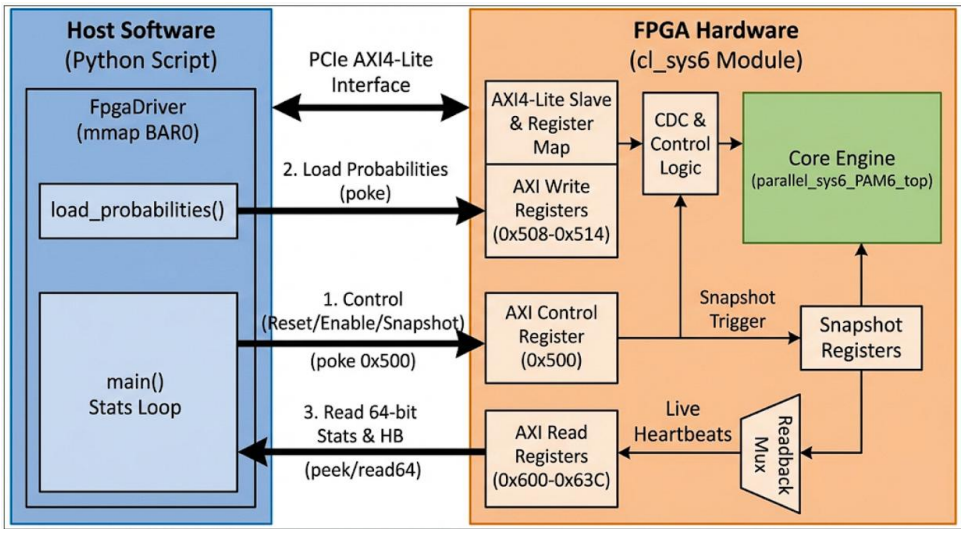


Figure 4: *Module-level schematic of the AWS platform software host.*

2.3.2 RTL Module-level Description (author: S. L. Kwan)

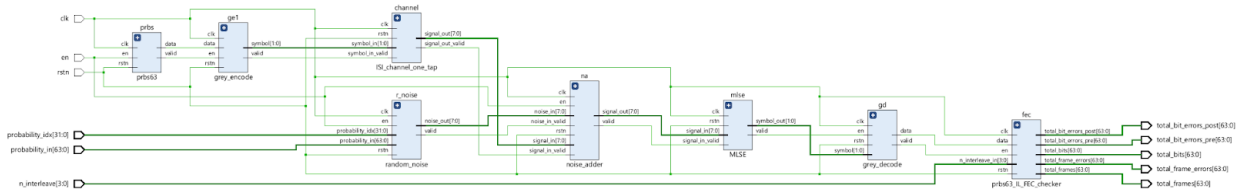


Figure 5: *Detailed block diagram of System 6 obtained from Xilinx Vivado.*

The hardware model implemented for performance characterization is System 6, which mimics the behavior of a single-part wireline link with $1 + \alpha D$ ISI channel impaired by additive white Gaussian noise (AWGN). Figure 5 illustrates the module-level block diagram consisting of eight components. Common components across all systems are PRBS, gray encoding, noise adder, gray decode, and forward error correction. Referencing from Barrie's paper [11], components specific to System 6 are as follows:

- ISI channel (ISI_channel_one_tap): uses a look-up table to pre-calculate the $1 + \alpha D$ response, and uses the table to output the deterministic ISI component in a single clock cycle
- Random noise (random_noise): uses a 64-bit Tausworth generator to implement a uniform random number generator, the 64-bit random word is used to index into a pre-programmed cumulative distribution function table. This converts the uniform distribution into an AWGN profile by selecting a noise magnitude and assigning a random sign bit
- Maximum Likelihood Sequence Estimator (MLSE): implements the Viterbi algorithm to find the most likely sequence of transmitted symbols through a six-state trellis. The module calculates squared Euclidean distance between the received signal and the ideal y-table levels, updating survivor path metrics and performing a traceback of length $L = 10$ to recover the symbols

This system is chosen specifically to verify the accuracy of the PAM-6 implementation, due to the computational complexity of the MLSE algorithm. Ensuring the accuracy of this system provides a baseline for other systems to be implemented. The highly modular system enables the swapping of channel models and equalization blocks. The two other systems brought up to AWS consist of the following differences: System 4 simplifies the link to a memoryless AWGN channel and introduces a hard-decision Hamming (128,120) inner FEC code to evaluate concatenated error correction. System 5 replaces the AWGN noise with analog low pass filter colored noise and includes additional blocks such as a FIR filter and convolutional inter-leaver to handle correlated noise. With the modular architecture, different configurations can isolate the impact of specific hardware blocks for final BER analysis.

2.4 Assessment of Final Design (author: J. Yang)

The following assessment provides a technical evaluation of the implemented platform against project requirements and performance targets. The final system fully meets the system requirements specification (SyRS), with more details explained in Section 3.

2.4.1 Speedup & Scalability

The core motivation for this project was to resolve the "validation crisis" where software simulations are too slow to guarantee 400G Ethernet reliability. While software models require more than years to encounter a single bit error at a 10^{-11} rate, our hardware-accelerated approach provides several orders of magnitude improvement [10]. Our design provides a single core 570x speed-up against traditional software simulation. Given the capability of the AWS FPGA, we can utilize spatial parallelism by instantiating multiple cores, i.e. duplicates of our simulator, which provides the scalability. With 100 cores, the speedup will be more than 57,000x. Section 3.3 explains the performance in greater detail.

2.4.2 Correctness

The system underwent a two-phase validation to ensure the hardware provides accurate and meaningful results. Two tests were performed to cross-check the correctness of the design.

1. The expanded PAM-6 implementation of System 6 was rigorously validated with a custom Python software reference model. By performing bit-by-bit comparisons of the PRBS generator output and the final bitstream, we confirmed a 0% mismatch.
2. To make sure that the AWS platform does not introduce further errors, the PAM-4 reference design was reproduced in the cloud and compared against.

These two tests have proved the correctness of our channel simulator in the cloud, with more details explained in Section 3.2.3 and 3.2.4.

2.4.3 Flexibility

The platform is engineered with high modularity to support diverse research needs. It currently supports three distinct system variants, ranging from simple links with analog AWGN channels to complex architecture utilizing MLSE equalizers and hard-decision Hamming codes. The RTL design is partitioned into reusable blocks for the transmitter (TX), channel model, and receiver (RX), enabling future researchers to swap specific modules, such as new 2D Reed-Solomon FEC blocks [12] or even PAM-8 modulation [8]. This flexibility is enhanced by the cloud environment, which eliminates the need for configuring the FPGA design differently on different local platforms.

2.4.4 User Friendliness

The comprehensive user interface and automation suite ensure long-term utility for the research community. The GUI abstracts the underlying AWS complexity, allowing users to trigger automated SNR sweeps and view live plots of simulation results. Figure 6 shows a screenshot of our GUI design.

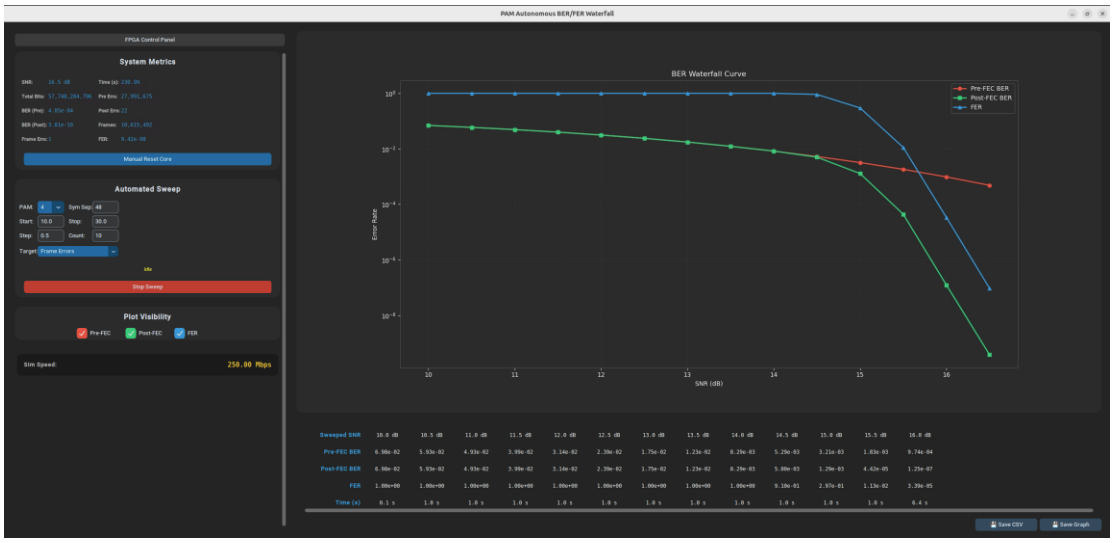


Figure 6: *AWS GUI interface display.*

Moreover, detailed instructions [16] and video tutorials [17] were made to help users quickly learn and deploy our design. The process was also simplified by helper scripts that shortened the steps compared to official AWS documentation. The materials were also tested by the group members to verify correctness.

2.4.5 Trade-offs

While the platform achieves significant throughput gains, there is a trade-off between execution speed and development time. There is substantial time required for hardware synthesis and implementation. As the design scales, specifically when increasing the number of hardware cores to maximize parallelism, the synthesis time in Vivado increases exponentially. In comparison, software simulation requires less effort or time to set up. In short, compiling time in preparation steps was sacrificed to enable fast runtime speed.

2.4.6 Possible Future Improvements

Future iterations of the platform could mitigate the limitations by expanding runtime configurability. Currently, runtime configurability is limited to a few parameters, such as noise levels and interleaving depths. To improve the system's flexibility, future versions should implement a more robust set of runtime-configurable registers. This could include the ability to switch between different PAM-6 constellation mappings or modify filter coefficients without requiring a full hardware re-compile.

2.4.7 Cost Analysis

Table 2 provides a cost analysis across different platforms for performing the channel simulation. Cost per one trillion bits of simulation was used as a metric to evaluate the cost of different platforms.

Table 2: *Cost analysis across platforms (all costs are US dollars)*

Platform	**Speed (Mb/s)**	**Upfront Cost**	**Hourly Cost (Estimated)**	**Cost per 1T (10^{12}) bits**
Software	0.219	$0	$0.30	$380.52
Vivado RTL	0.052	$0	$0.30	$1,602.56
Opal Kelly XEM7310	40	$630	$0	-
AMD ZCU102	1200 [11]	$3,300	$0	-
AWS	125 * N_cores	$0	$3.00	$6.67 / N_cores

Traditional software simulation and Vivado RTL simulation run on CPUs; as a result, they have low simulation speed. Their hourly cost was estimated using a normal performance machine on AWS with 4 CPU cores and 16GB memory – $0.20/hour plus $0.10/hour overhead for disk usage and network traffic.

The Opal Kelly XEM7310 FPGA board was used for our feasibility test (described more in Section 3.2.1). It is a relatively low-performance board, thus having lower speed and price. In comparison, the AMD ZCU102 is a high-performance local board which was used in the reference design [11]. It has a higher price and higher speed than the Opal Kelly board due to more advanced manufacturing techniques and more resources on the chip. In short, the local FPGA boards have non-negligible upfront costs that are related to their performance, but no hourly cost.

Finally, the AWS platform has no upfront cost. Although the hourly cost is higher than CPU-based platforms, this difference is offset by the massive speed increase. The speed is also directly proportional to the number of cores (N_cores) instantiated for parallelism, resulting in only $6.67 per 1T bits for a single core, and, for example, having 10 cores will reduce to only $0.67 per 1T bits while having similar simulation speed compared to the ZCU102 board. This indicates that the AWS cloud solution has a cost advantage in addition to the other benefits described above.

3. Testing and Verification

This section elaborates on how the testing and verification process proved that our design has met the system requirements.

3.1 Verification Table (author: J. Yang)

Table 3 lists the requirements for the project, provides the verification method, and indicates test results.

Table 3: *System requirements with verification methods and proof*

ID	Specification	Verification Method	Verification Result and Proof
1	The system must support BER analysis of PAM6 modulation under configurable noise conditions.	TEST: Apply PRBS63 data through the PAM6 channel. Adjust noise parameters and detect changes in BER.	PASS, See Section 3.3
2	The platform must provide modular support for alternative FEC/Receiver architectures.	TEST: Replace FEC/Receiver module (e.g., MLSE or hard slicer) and re-synthesize. Verify correct decoding and identical host-interface behavior.	PASS, See Section 3.3
3	The system must have a software baseline model that verifies the correctness of the RTL model	ANALYSIS: Compare RTL simulation results to Python reference outputs; <0.1% relative difference should be observed	PASS, See Section 3.2.3
4	The system must have faster single-core speed than the Opal Kelly XEM7310 board	TEST: Execute identical symbol count on FPGA (AWS); verify throughput compared to Opal Kelly.	PASS, See Sections 3.2.1, 3.3
5	The system must support PAM4 modulation as a baseline to verify correctness against the reference design	TEST: Run PAM4 mode on AWS; reproduce reference BER plots. Matching results would confirm the correct implementation.	PASS, See Section 3.2.4
6	The system must log key performance statistics (pre-FEC BER, post-FEC BER, FER) to files for later analysis	TEST: Run SNR sweep in simulation; confirm simulation statistics can be generated and exported as .csv files	PASS, See Section 3.3
7	The system must include a host PC control interface	TEST: Use a Python or C host application to read/write FPGA registers via AWS Shell interface. Confirm that parameters can be	PASS, See Section 3.3

		successfully sent, and error counters can be correctly read.	
8	The system must support parallel-core execution to enable scalability	TEST: Enable multi-core mode (by changing the parameter N_CORES); verify near-linear throughput scaling (±1% per added core).	PASS, See Section 3.3
9	The system must support runtime configuration of channel parameters (e.g., SNR, precoding, interleaving) without resynthesis	TEST: Modify parameters through host registers during operation; confirm immediate change in output BER trend.	PASS, See Section 3.3
10	The system must provide sufficient documentation of APIs and configuration steps to enable use by external users	TEST: Follow the documentation to verify that the design can be successfully reproduced.	PASS, See Section 3.3

3.2 Detailed Testing Procedures

The testing procedure is split into four distinct parts: a feasibility testing on a local FPGA board; hardware verification through Vivado behavioural Simulation on Vivado; software modelling & benchmarking; AWS platform implementation. Each step is elaborated in the following sections.

3.2.1 Feasibility Testing on a local FPGA board (author: E.T. Sumual)

The initial phase of the project focused on a hardware-accelerated proof of concept that determines the local FPGA board performance and serves the baseline for Requirement #4. Opal Kelly XEM7310 board was chosen, given that it has robust software API and pre-built hardware Verilog IP blocks which makes it easy to set up [18]. This ecosystem testing ensured our proposed architecture was functionally viable on a local board before committing to a more complex, cloud-based deployment.

The system achieved a stable throughput of 40 Mbps. When compared to the software simulation speed of 0.219 Mbps, this represents a 182x speedup. By proving this level of hardware acceleration locally, we established the necessary baseline performance required to justify moving to a scalable, faster-than-local cloud hardware solution on AWS.

Appendix D described this test in greater detail, which includes (1) a block diagram of the setup, (2) explanation of the software-hardware interface, and (3) terminal output from the testing

3.2.2 Hardware Model Verification in Vivado (author: E.T. Sumual)

Following the initial feasibility tests, the focus shifted to ensuring the mathematical and functional accuracy of the hardware through RTL simulation in Vivado. The primary goal of this stage was to verify that the Verilog implementation of our signal processing chain, specifically the transition to PAM6 modulation, maintained integrity across all modules before deployment in the cloud.

We conducted tests to monitor signal propagation through the system in Vivado. This simulation allowed us to verify PAM-6 logic for all systems, ensure that the conversion from PAM-4 to PAM-6 correctly mapped the higher-order modulation symbols.

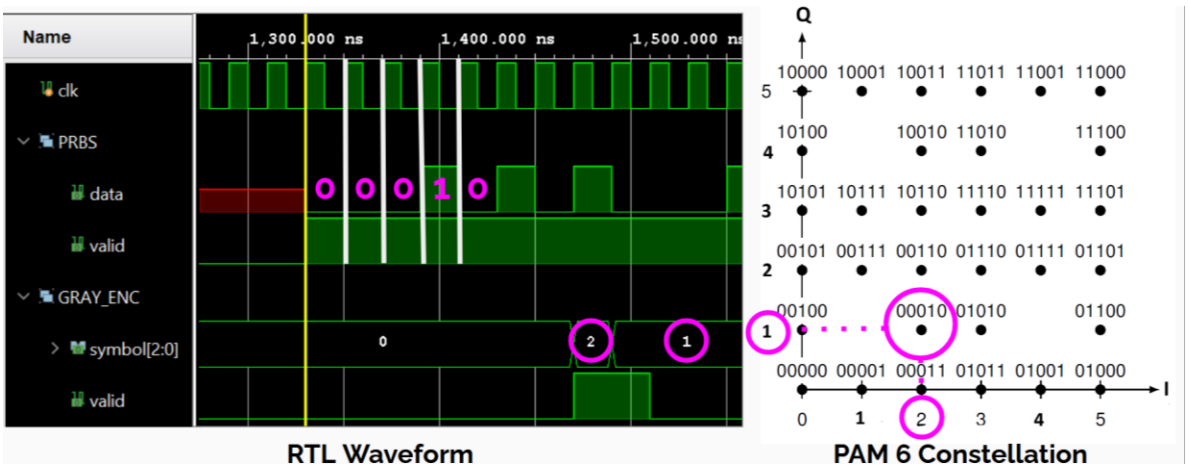


Figure 7: *RTL Signal Verification vs. PAM-6 Constellation [5].*

A critical component of our accuracy testing was the verification of the gray encoder (GRAY_ENC) to ensure that digital bit streams were correctly mapped to their corresponding physical symbols. As shown in Figure 7, we traced specific bit patterns from the PRBS generator through the encoding logic. By isolating a specific sequence, for example, the binary pattern 00010, we validated that the hardware correctly identified the target symbol in the 2D PAM-6 constellation space (represented as point 2 on the I-axis and 1 on the Q-axis). This bit-to-symbol verification confirmed that symbols were mapped to the correct adjacent constellation points.

This level of granular signal verification provided the expected behavior for the hardware. It ensured that any errors observed later during the AWS bring-up would be a result of channel noise or cloud configuration, rather than fundamental logic errors in the modulation scheme.

3.2.3 Software Reference Model & Testing (author: S. L. Kwan)

The software model was implemented in Python, with the main data path implementation described in Appendix E. To ensure the software model accuracy for System 6, three tests were designed and executed:

(1) comparison of RTL with software at each stage from PRBS to the pre-FEC block

(2) comparison of post-FEC outputs between RTL and reference model

(3) comparison with AWS implementation outputs

```
$ python sys6_model.py
[RTL-PRBS vs Ref-PRBS] Compared 999933 bits (offset 0). Mismatches: 0  BER=0.000e+00
[RTL-PostGE vs Ref-PostGE] Compared 399971 bits (offset 0). Mismatches: 0  BER=0.000e+00
[RTL-PostISI vs Ref-PostISI] Compared 399970 bits (offset 0). Mismatches: 0  BER=0.000e+00
[RTL-PostMLSE vs Ref-PostMLSE] Compared 399959 bits (offset 0). Mismatches: 0  BER=0.000e+00
[RTL-PreFEC vs Ref-PreFEC] Compared 999890 bits (offset 0). Mismatches: 0  BER=0.000e+00
Reference simulation time: 6.06 seconds
```

Figure 8: *One-million-bit comparison between RTL and reference model for pre-FEC stages.*

Figure 8 shows the results of (1), one million bits were compared to ensure data path accuracy at high volume input. RTL data was extracted during behavioral simulation into a file output. The software reference model compared its own output with the extracted data after completing the post-FEC stage, yielding the results shown above. A mismatch count was embedded in the software model; a zero mismatch indicates a bit-true model is achieved, as shown in the figure.

```
RUNNING SIMULATION: 15 dB
Using noise memory: noise15dB_sep24.mem
[[15dB] RTL-PRBS vs Ref-PRBS] Compared 999933 bits (offset 0). Mismatches: 0  BER=0.000e+00
[[15dB] RTL-PreFEC vs Ref-PreFEC] Compared 999890 bits (offset 0). Mismatches: 0  BER=0.000e+00

Metric                    | RTL Value       | Python Ref      | Match?
-------------------------------------------------------------------------------
total_bits                | 999889          | 999970          | ✅
total_bit_errors_pre      | 46055           | 46061           | ✅
total_bit_errors_post     | 45879           | 45887           | ✅
total_frames              | 183             | 183             | ✅
total_frame_errors        | 183             | 183             | ✅
BER_pre                   | 4.606e-02       | 4.606e-02       | ✅
BER_post                  | 4.588e-02       | 4.589e-02       | ✅
Simulation time: 6.78 seconds
```

Figure 9: *Matching FEC checker output results given system with noise input.*

Figure 9 shows the results of (2), approximately one million pre-FEC and post-FEC bits were compared to test the accuracy of the outer FEC algorithm and output statistics generation (prbs63_IL_FEC_checker). Due to hardware pipeline slack during high-volume simulation, the output

values were compared with 0.1% relative tolerance instead of zero. The test results matched RTL implementation within the tolerance level, proving model accuracy.

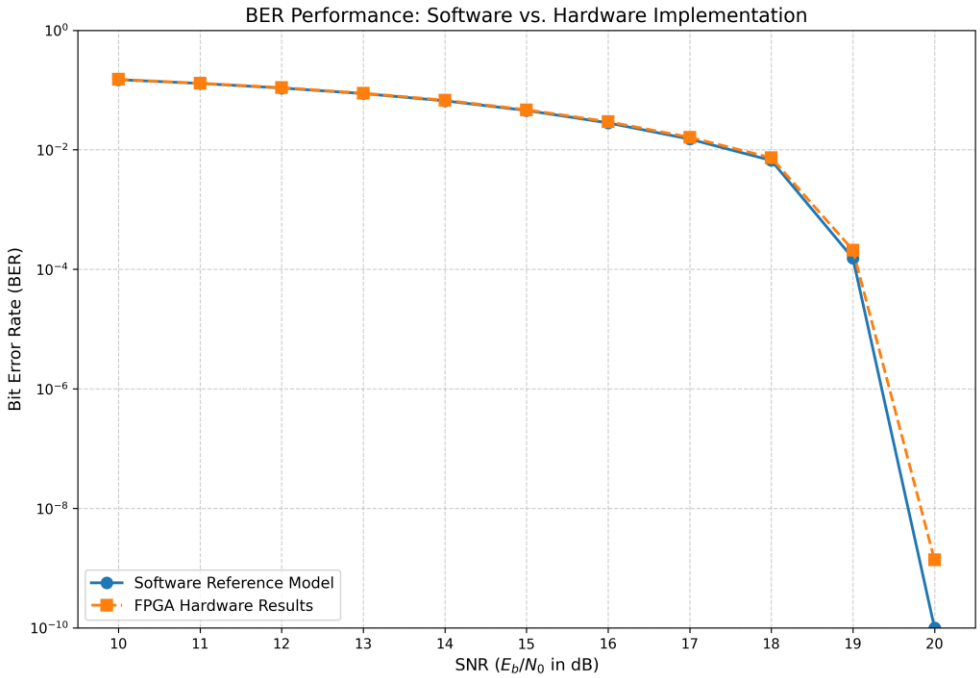


Figure 10: *Plot comparing FEC statistics in software and AWS FPGA Hardware implementation.*

Figure 10 shows the results of (3). The software model was tested with 10dB to 20dB signal-to-noise (SNR) ratio, and the final BER statistics (`BER_pre`, `BER_post`) were collected when the total post-FEC bit error count (`total_bit_errors_post`) reached 100k bits. The error count limit was obtained by experimentation as it is observed that the BER accuracy is achieved by having high volume output. The figure shows matching results for 10dB to 19dB SNR, with a deviation at 20dB. The error rate statistic for 20dB could not be achieved for the software model, because no errors were simulated after a 50-hour runtime. In contrast, the AWS platform was able to generate reasonable BER output to give its much higher simulation speed. This again demonstrated that software simulation was unable to run for the time required to generate positive bit error count, supporting the motivation that hardware acceleration is needed for channel simulation.

3.2.4 AWS Setup Verification (author: J. Yang)

Given that the cloud-based FPGA is abstracted behind the designed AWS Shell and interfaces with the host CPU via AXI4-Lite protocols, it was necessary to prove that these communication layers and the necessary CDC synchronizers did not corrupt the simulation. Thus, to ensure that the transition from a local development environment to a cloud infrastructure did not introduce architectural or numerical errors, a test was designed and executed. Given that the reference design is open-source, and that the BER-SNR results are available, the idea was to bring one of the designs to the AWS platform and then compare the generated result curve with the reference work.

The PAM-4 reference design for System 6 by Barrie was reproduced and migrated to the AWS environment. A BER-SNR sweep was performed to generate the waterfall curve, which was compared against the curve in the reference design. As expected, the two curves overlapped, which proved that the cloud infrastructure did not introduce extra error. Figure 11 shows the two curves overlapping.

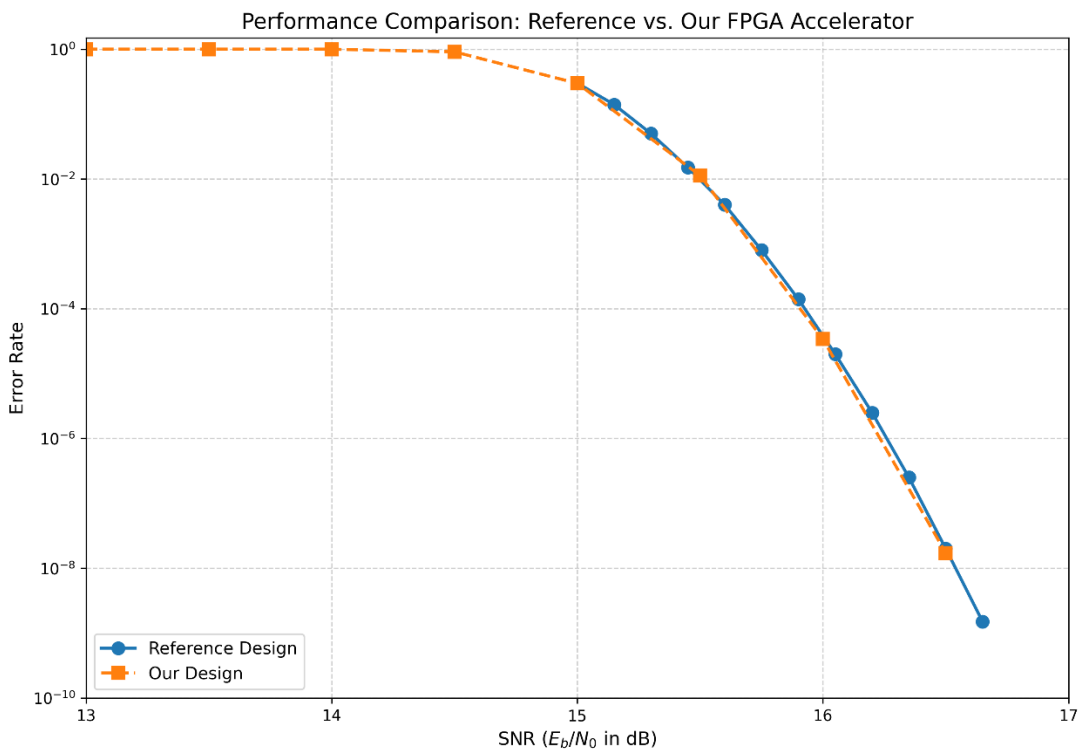


Figure 11: *BER-SNR sweep of reference model and AWS cluster implementation.*

3.3 Final Test Results (author: E. T. Sumual)

The testing and verification conducted across local FPGA hardware, RTL simulation, software modeling, and AWS cloud deployment has successfully validated all system specifications. By cross-validating between software reference models and hardware models, we have confirmed that the system is both mathematically accurate and meets the throughput requirements for 400G Ethernet.

3.3.1 Summary of Requirement Fulfillment

The following results summarize the final status of the specifications outlined in the Verification Table:

- **Accuracy and Mathematical Parity** (**Requirement ID #3, #5**): through extensive comparison between the Python reference model and Vivado RTL simulations, we have validated the accuracy of our hardware model. The overlapping FER-SNR waterfall curves between the AWS-based System 6 and the reference design by Richard Barrie confirm that no architectural or numerical errors were introduced during the transition to the AWS platform.
- **Modulation & Control (#1, #2, #7, #9**): the system fully supports PAM-6 modulation and runtime configuration (SNR, noise) via the host PC interface without requiring re-synthesis.
- **Acceleration & Scalability** (#**4, #8**): the project delivers a 3.1x throughput increase via the AWS implementation (125 Mb/s) over the local Opal Kelly XEM7310 baseline (40 Mb/s) and demonstrates near-linear scaling, reaching 250 Mb/s in a dual-core configuration.
- **Performance Monitoring (#6, #10):** final testing confirmed that all key statistics (pre-FEC BER, post-FEC BER, and FER) can be logged to the host PC in either graph or .csv format. This ensures a high-fidelity verification platform for external users.

3.3.2 Throughput and Speedup Comparison

Table 4 below highlights the performance gain achieved through parallel-core execution on AWS.

Table 4: *Throughput and speedup data for each platform*

Platform	**Throughput (Mb/s)**	**Speedup**
Python Model	0.219	1x (baseline)
Vivado RTL Simulation	0.052	0.24x
Local FPGA Board (Opal Kelly XEM7310)	40	182x
AWS (Our Design): 1 Core	125	570x
AWS (Our Design): 100 Cores	12500	57,000x

4.0 Summary and Conclusions (author: K. Jiao)

This section summarizes the project and analyzes its usefulness and possible future applications.

4.1 Project Summary and Design Validation

The core objective of this project was to resolve the "Validation Crisis" in modern Ethernet development, where traditional software simulations are impractically slow for guaranteeing the reliability of 400G networks [10]. We successfully addressed this by delivering a highly scalable, cloud-enabled FPGA acceleration platform deployed on AWS.

The final design successfully met all system requirements within the SyRS. By transitioning the hardware models from 200G PAM-4 to 400G PAM-6 architectures, the system achieved throughput necessary for high-speed Ethernet architecture exploration and 400G performance benchmarks. Validation tests demonstrated a 570x simulation speedup on a single AWS core compared to baseline software models, and the possibility of even higher speed up when utilizing spatial parallelism.

Furthermore, the project was successfully delivered within the defined constraints of our financial plan by efficiently managing AWS instance costs and utilizing local Opal Kelly hardware for initial feasibility testing. Based on the project timeline, the project implementation was prioritized for Systems 4, 5, and 6. This selection allowed the platform to address alternative encoding schemes as well as FEC and receiver architectures.

4.2 Comprehensive Model Verification

While hardware acceleration is essential for overcoming the time constraints of software models, the results from hardware acceleration are valuable if and only if the model is bit-accurate. Thus, significant emphasis was placed on cross validating the system across multiple environments to ensure the generated errors are accurate and useful for research.

This verification was achieved through a multi-stage process:

- **RTL Behavioral Simulations**: we used Vivado to simulate signal propagation and precisely verify the digital bit-to-symbol mapping for the PAM-6 QAM-32 constellation grid.
- **Python Reference Modeling**: we developed a bit-accurate Python model to cross-check the hardware. By comparing a million bits across pre-FEC and post-FEC stages, we achieved near a zero-mismatch tolerance, proving the modelling accuracy of the MLSE and FEC algorithms.

- **AWS-FPGA Sweeps**: we performed automated BER-SNR parameter sweeps on the EC2 F2 instance and compared the generated BER-SNR plots against Richard's baseline PAM-4 design [11]. The alignment of these curves verified that the AWS clock-domain crossing and custom logic wrappers maintains data integrity without logic errors.

4.3 Usefulness in Industry and Academia

By bridging the gap between slow software model and fast hardware execution, this platform functions as a research and validation tool for high-speed communications. The user-friendly interface created is especially useful in academia, as the inclusion of a GUI and automation scripts allows researchers to evaluate new FEC and receiver architectures by abstracting the underlying hardware complexity. In industry, the platform enables semiconductor designers to characterize wireline transceiver performance across diverse channel conditions prior to silicon fabrication. This provides a reusable framework for verifying the robustness of 400G Ethernet standards, ultimately reducing the computation time required for deep BER analysis.

4.4 Future Work and Applications

While the current platform is highly capable, there are still a wide range of future expansion options:

- **More Modulation Schemes**: The system is currently tested for PAM-4 and PAM-6. Future work should focus on expanding the modulation mappers and slicers to support PAM-8, which has been identified as a strong candidate for future 400G and 800G Ethernet standards [8] [13].
- **Integration of New Algorithmic Blocks**: The platform's modularity allows researchers to easily swap channel models and equalization blocks. Future iterations should integrate more complex algorithmic blocks, such as 2D Reed-Solomon FEC [11].
- **Industry Application**: The platform offers a framework that aligns with the development cycles of semiconductor and electronic design automation (EDA) firms. Integrating our project into existing development cycle could facilitate faster characterization for next-generation hardware and provide the project with valuable industry input.

5. References

[1] Microsoft, "NP family VM size series - Azure Virtual Machines," Microsoft Learn, June 6, 2024. [Online]. Available: https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/fpga-accelerated/np-family. [Accessed: Mar. 21, 2026].

[2] Optical Internetworking Forum (OIF), CEI-224G Framework Document, Version 01.0, Feb. 2022. [Online]. Available: https://www.oiforum.com/specifications/

[3] T. Carusone, "425–448 Gbps Electrical Feasibility for C2C and C2M using COM," IEEE 802.3 E4AI Ad Hoc Meeting, May 29, 2025. [Online]. Available: https://www.ieee802.org/3/ad_hoc/E4AI/public/25_0529/carusone_e4ai_01a_250529.pdf

[4] A. Ran, "Proposal for BER Budget Allocation for AUIs in Type 1 and Type 2 PHYs," IEEE P802.3dj Task Force, May 2023.

[5] G. Böcherer, "On the Performance of PAM-6 Modulation Schemes for High-Speed Ethernet," arXiv preprint arXiv:2205.05460, May 2022. [Online]. Available: https://arxiv.org/pdf/2205.05460

[6] J. Wei et al., "Experimental demonstration of advanced modulation formats for data-center networks on 200 Gbps lane-rate IM/DD links," Opt. Express, vol. 28, no. 23, pp. 35240–35256, Nov. 2020.

[7] J. Tang et al., "O-band EML and 4-bit DAC enabled 112 GBaud PAM-8 transmission over 10 km SSMF without optical amplifiers," Opt. Lett., vol. 49, no. 23, pp. 6605–6608, Nov. 2024.

[8] C. Xu, "400 Gbps+ per Lane Transmission with PAM-4, 6, 8," IEEE 802.3 E4AI Ad Hoc Meeting, Jun. 24, 2025. [Online]. Available: https://www.ieee802.org/3/ad_hoc/E4AI/public/25_0624/xu_e4ai_01a_250624.pdf

[9] J. He, "Low Latency Mode for Inner FEC," IEEE P802.3dj Task Force, Jan. 2024.

[10] R. Gustlin, "PCS, FEC and PMA Overview (802.3bs/ck)," IEEE 802.3db Ad Hoc, Jun. 2020.

[11] R. Barrie, Analysis of Wireline FEC Performance Using an FPGA Platform, M.A.Sc. thesis, Dept. Elect. & Comput. Eng., Univ. of Toronto, 2024.

[12] J. Cirit, "448G/Lane Modulation & FEC," IEEE 802.3 E4AI Ad Hoc Meeting, May 29, 2025. [Online]. Available: https://www.ieee802.org/3/ad_hoc/E4AI/public/25_0529/cirit_e4ai_01a_250529.pdf

[13] M. Graumann, “PAMXmY Modulation Proposal for 448G,” IEEE 802.3 E4AI Ad Hoc Meeting, Jun. 24, 2025. [Online]. Available: https://www.ieee802.org/3/ad_hoc/E4AI/public/25_0624/graumann_e4ai_01a_250624.pdf

[14] J. Cosson-Martin, K. Zheng, S. Kiran, A. Ramkaj, R. Casey, H. Zhang, J. Im, Y. Frans, and G. Zhang, “400G+ Electrical Pathfinding,” in Proc. DesignCon 2025: Where the Chip Meets the Board, Santa Clara, CA, USA, Jan. 2025, presented by AMD

[15] Amazon Web Services, “Amazon EC2 F2 Instances,” [Online]. Available: https://aws.amazon.com/ec2/instance-types/f2/. [Accessed: Mar. 21, 2026]

[16] Capstone 148. “Manual for Running the Design on AWS EC2 F2 Instance.” Notion, Mar. 2026, brett-yang.notion.site/Manual-for-running-the-design-on-AWS-EC2-F2-instance-2e7f48807ff580379cc0ef8710af2647. [Accessed: Mar. 21, 2026]

[17] Capstone 148, “AWS FPGA Walkthrough,” Google Docs, Feb. 26, 2026. Available: https://drive.google.com/file/d/1dAEO2YmVMeRsG_TutIWvumbxLFbet8KS/view?usp=sharing. [Accessed: Mar. 21, 2026]

[18] Opal Kelly, “FrontPanel® - Opal Kelly,” Opalkelly.com, 2025. Available: https://www.opalkelly.com/products/frontpanel/?srsltid=AfmBOoruH8he1YqySNOPt2ESRzZ7oow08lwMiKIFo__R9IJjpRCv4PVP. [Accessed: Mar. 21, 2026]

[19] Opal Kelly, “XEM7310 - Artix-7 FPGA Development Board,” 2026. [Online]. Available: https://www.opalkelly.com/products/fpga-integration/XEM7310/

6. Appendices

Appendix A. Gantt Chart History (author: S. L. Kwan)

The Gantt Chart (Figure A1) has been updated to reflect individual contributions. Key differences with Figure A2 from implementation plan include (1) significant overlap between implementation, testing, and reporting phases due to iterative debugging (2) reduction of system design and implementation to from all nine systems in Richard's PAM4 models to three (sys 4, 5, 6) to ensure system 6 functionality, (3) addition of tasks such as AWS GUI, documentation, and validation.

Figure A3 shows the original Gantt Chart from Proposal Document.

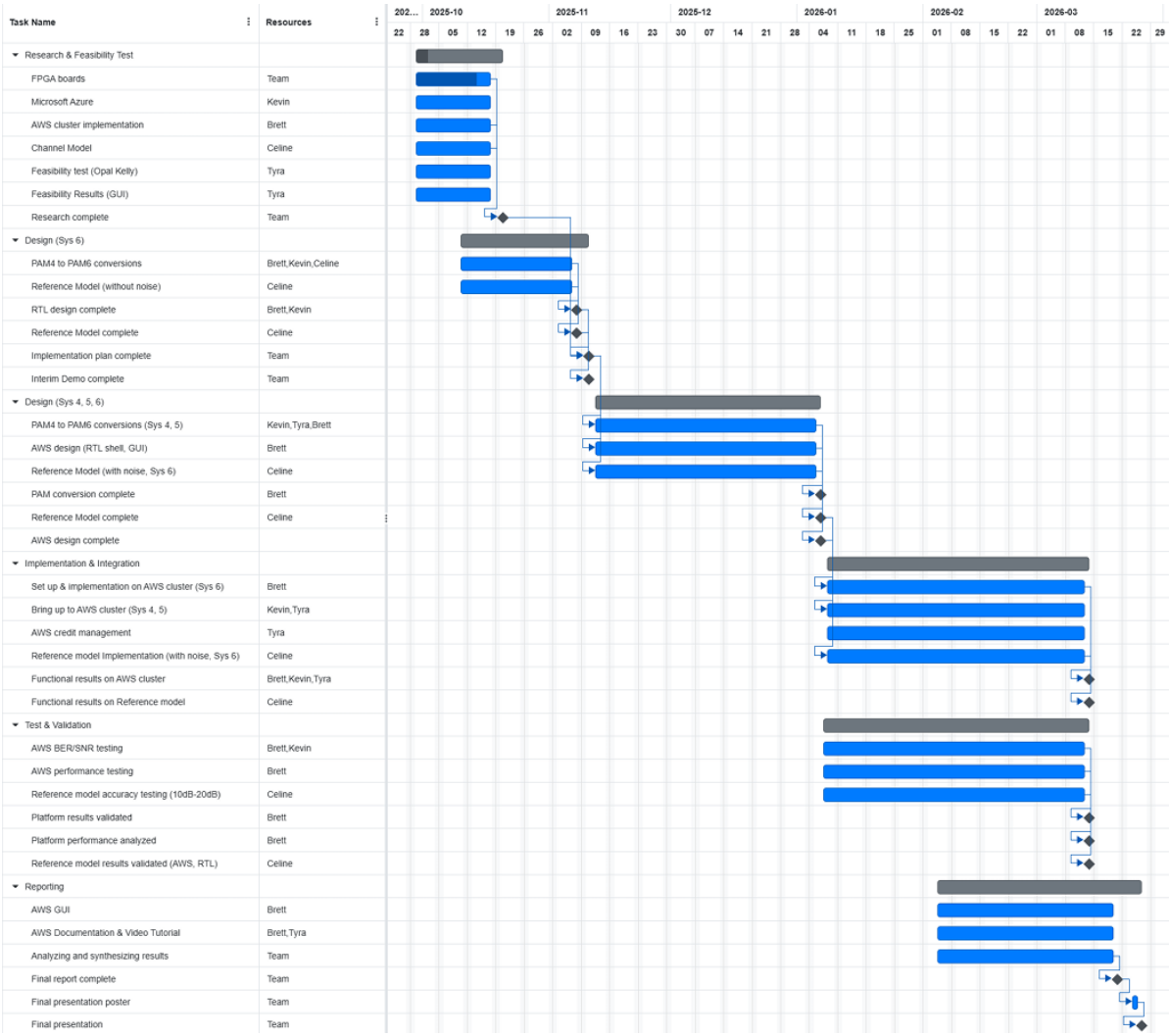


Figure A1: *Final version of Gantt Chart to reflect work after implementation plan.*

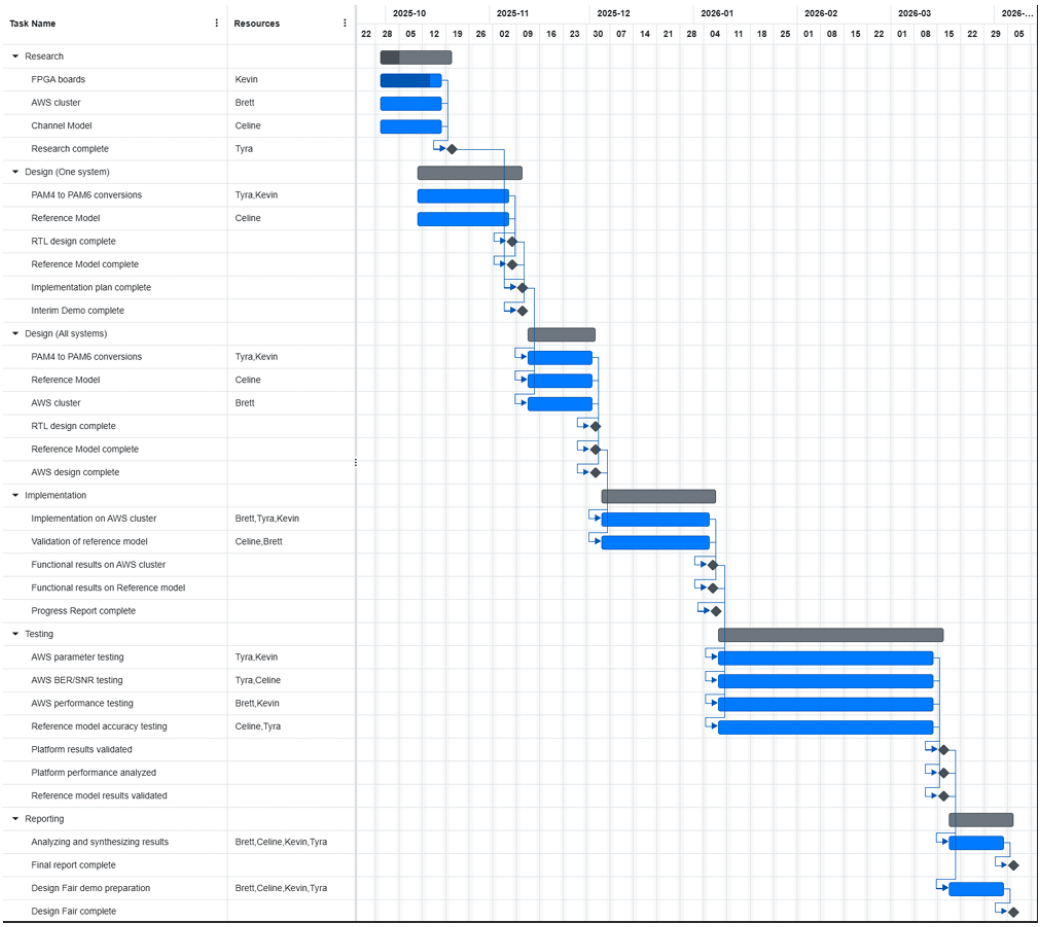


Figure A2: *Gantt Chart Version 2 from implementation plan.*

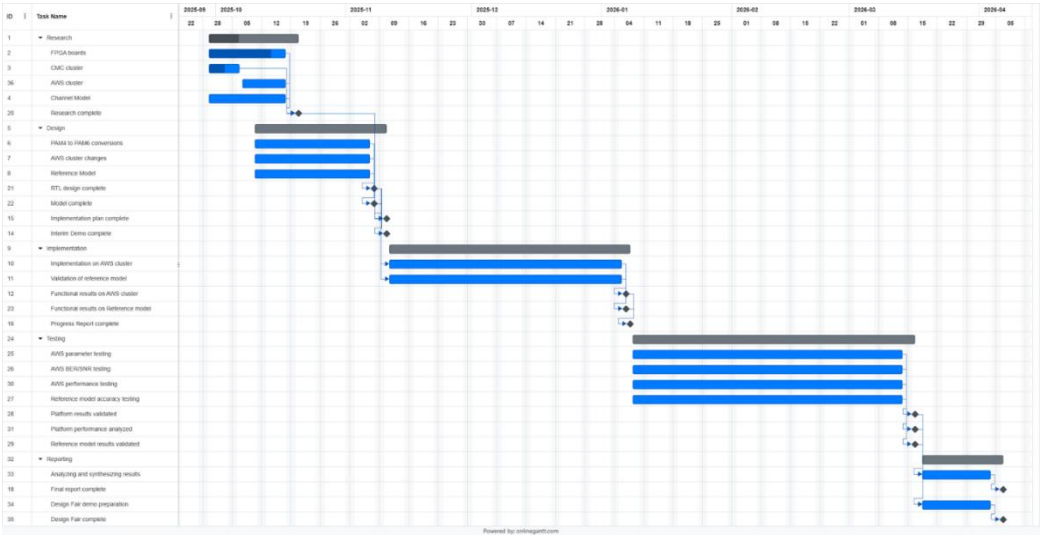


Figure A3: *Gantt Chart Version 1 from Proposal Report.*

Appendix B. Financial Plan (author: J. Yang)

Figure B1 summarizes the budget for this project, including services, capital equipment, and student labour.

Each team member has registered for the $120 USD AWS free credit, resulting in a total of $480 USD credits to be used. The team completed all design and testing under the credit, so no cost was induced.

Consumables/Services								
Item	**Priority**	**Cost/unit [USD]**	**Quantity (# of hours)**	**Total Cost [USD]**	**Total Cost [CAD] *$1 USD = $1.4 CAD**	**Requires Funding**		**Notes**
AWS - m4.2xlarge	1	$0.40/hr	100	$40.00	$56.00	N		for design synthesis & FPGA AWS AFI generation
AWS - f2.6xlarge	1	$1.98/hr	100	$200.00	$280.00	N		for actualy BER testing on the cloud FPGA
AWS - storage	1	$0.10/ (GB*month)	200GB * 4 months	$80.00	$112.00	N		
Github	1	Free	N/A	$0	$0	N		
Vivado license	1	Free	N/A	$0	$0	N		included in AWS FPGA AMI
AWS Free Credit	1	-$120/person	4 persons	-$480	-$672	N		
Total Consumables / Services					**$0**			
Total Requiring Funding					**$0**			
Capital Equipment								
Item	**Priority**	**Cost/unit [USD]**	**Quantity**	**Total Cost [USD]**	**Total Cost [CAD]***	**Requires Funding**		**Notes**
Opal Kelly XEM8305	1	$759.95/board	0	$0	$0	N		Backup plan, purchase only if AWS does not work
Total Capital Equipment					$0			
Total Requiring Funding					$0			
Student Labour					**Student Labour Hour Breakdown**			
Item	**Cost/unit**	**Quantity (# of hours)**	**Total Cost**		**Stage**	**Weeks**	**# of Hours/Week**	**Total Hours**
Jiaxin (Brett) Yang	$42.00	230	$9,660.00		Research	2	5	10
Suet Ling (Celine) Kwan	$42.00	230	$9,660.00		Design (One System)	3	15	45
Kevin Jiao	$42.00	230	$9,660.00		Design (All Systems)	2	5	10
Elizabeth Tyra Sumual	$42.00	230	$9,660.00		Implementation	3	20	60
Total Student Labour (unfunded)			**$38,640.00**		Testing	7	15	105
					Total Hours			**230**
Funding								
Students ($100 each)	$0							
Supervisor	$0							
Other (specify)	$0							
Requested from ECE496	$0							
Total Funding	$0							

Figure B1: *Project Budget Table.*

Appendix C. Explanation of Technical Background (author: All)

Wireline communication refers to data transmission over a guided medium, such as electrical interconnects or optical fibers, where the signal travels along a physical path. Ethernet is a family of wireline communication standards that define how digital data is transmitted, defined by IEEE. It consists of the physical layer, which handles encoding of bits into electrical or optical signals, and the data link layer, which handles data framing and error correction. Modern Ethernet links face challenges such as signal attenuation (reduction of signal strength), inter-symbol interference (**ISI**, corrupting of symbols transmitted), and channel loss (reduction of signal power at each frequency). Currently, industry research focuses on improving speed (measured by Gbps) whilst maintaining signal integrity (reducing signal degradation such as noise distortions). Most recent developments on Ethernet focus on increasing speed from 100 Gbps to 200 Gbps and 400 Gbps.

This introduces development areas to improve how signals are packed, and how errors are being corrected. Signal packing can be done via modulation schemes. Modulation describes how signals are varied to transmit information.

Modulation schemes that tie most to wireline communication are Pulse-Width modulation schemes, options available include PAM-4, PAM-6, or PAM-8, where the number indicates how many voltage levels are available for information transmission, and correlates to the number of bits per symbol (log2N). The benefit of having a higher N is increased throughput (transmitting more bits per symbol) — without this, bits travel at high frequencies and may exceed the channel bandwidth (range of frequencies a communication channel can support). Current research aims to explore higher-level modulation (higher N) to increase throughput, however this evidently introduces tradeoffs in noise and linearity (signal distortions).

In terms of error correction, with an increased number of voltage levels, there is less tolerance to noise and distortion due to closer distance between symbol thresholds. This is amongst several other issues introduced due to higher data rates — they could be addressed later during the project. In addition, with increased frequency, the signal weakens (attenuation), interference between electromagnetic waves increase - such as inductance and capacitance (crosstalk), and increased timing variations at signal edges (jitter), causing signals to be sampled at the wrong time. This introduces the need for forward-error correction (FEC) architectures.

Forward error correction is a mechanism in which errors are removed by, in fact, introducing more errors. In a wireline system with a transmitter (TX) and receiver (RX), parity bits (a bit added to check

correctness) are added to the TX signal. The simplest implementation is the parity bit that indicates whether the total number of 1's transmitted is odd or even — as the RX receives the signal, it counts the number of 1's and checks if the even or odd type matches. If it doesn't, an error occurred. Improved and more sophisticated FEC architectures include Hamming and Reed-Solomon error correction. Different types of codes exist to handle different amounts of errors — e.g. Hamming, Reed-solomon for large numbers of errors, simple FECs for less errors. Design options also include having multiple layers of error correction within the TX-RX system, such as having an inner layer (inner code) and outer layer (outer code) — inner codes can correct high frequency noise near the channel output and outer codes can provide further correction. Several options are considered in our project, and we aim to measure the performance of variations.

Appendix D. More Details on Feasibility Test on Opal Kelly Board (author: E. T. Sumual)

Figure D1 shows the block diagram of the hardware setup for the feasibility test on the local Opal Kelly XEM7310 board.

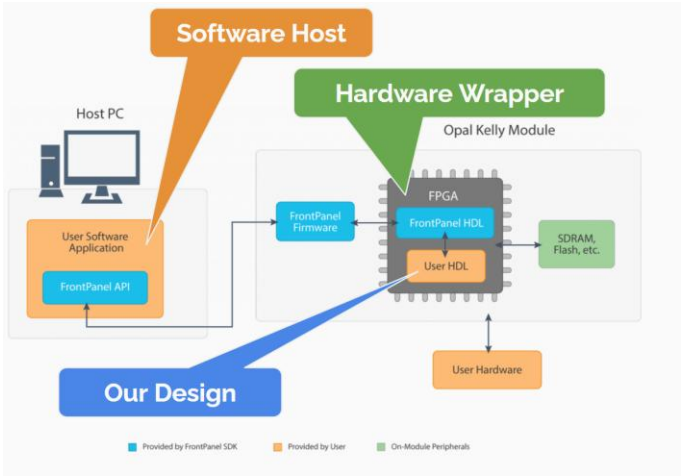
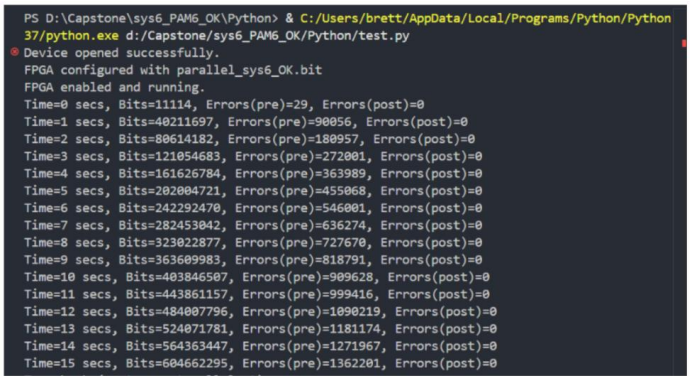


Figure D1: *Opal Kelly Testing Block Diagram [19].*

The testing architecture utilized the Opal Kelly FrontPanel SDK to bridge the communication between our user software and the FPGA hardware. A custom Verilog wrapper was implemented that integrated our System 6 modules (converted from PAM4 to PAM6) with the okHostInterface. This allowed the design to communicate with a Python-based host application developed to manage device configuration and record Pre-FEC and Post-FEC results.

Figure D2 shows the output of the hardware at a speed of 40Mb/s.



```
PS D:\Capstone\sys6_PAM6_OK\Python> & C:/Users/brett/AppData/Local/Programs/Python/Python
37/python.exe d:/Capstone/sys6_PAM6_OK/Python/test.py
Device opened successfully.
FPGA configured with parallel_sys6_OK.bit
FPGA enabled and running.
Time=0 secs, Bits=11114, Errors(pre)=29, Errors(post)=0
Time=1 secs, Bits=40211697, Errors(pre)=90056, Errors(post)=0
Time=2 secs, Bits=80614182, Errors(pre)=180957, Errors(post)=0
Time=3 secs, Bits=121054683, Errors(pre)=272001, Errors(post)=0
Time=4 secs, Bits=161626784, Errors(pre)=363989, Errors(post)=0
Time=5 secs, Bits=202004721, Errors(pre)=455068, Errors(post)=0
Time=6 secs, Bits=242292470, Errors(pre)=546001, Errors(post)=0
Time=7 secs, Bits=282453042, Errors(pre)=636274, Errors(post)=0
Time=8 secs, Bits=323022877, Errors(pre)=727670, Errors(post)=0
Time=9 secs, Bits=363609983, Errors(pre)=818791, Errors(post)=0
Time=10 secs, Bits=403846507, Errors(pre)=909628, Errors(post)=0
Time=11 secs, Bits=443861157, Errors(pre)=999416, Errors(post)=0
Time=12 secs, Bits=484007796, Errors(pre)=1090219, Errors(post)=0
Time=13 secs, Bits=524071781, Errors(pre)=1181174, Errors(post)=0
Time=14 secs, Bits=564363447, Errors(pre)=1271967, Errors(post)=0
Time=15 secs, Bits=604662295, Errors(pre)=1362201, Errors(post)=0
```

Figure D2: *Output from the local board, where simulation speed is 40 Mb/s.*

Appendix E. System 6 implementation in Python (author: S. L. Kwan)

Figure E1 is the code snippet from the software reference model aligns with RTL modules described in section 3.2.3.

```
 # Channel & Noise Injection
isi = ISIChannelOneTapPAM6(symbol_sep=24.0, alpha=0.5)
ch_out = isi.process(syms)
noise_file = f"noise{snr}dB_sep24.mem"
noisy = ch_out
valids = [True] * len(ch_out)

# RX Equalization & Demapper
mlse = MLSEPAM6(SYMBOL_SEPARATION=24, SIGNAL_RESOLUTION=8, ALPHA=0.5, TRACEBACK=10,
METRIC_RESOLUTION=20)
dec_syms = []
for i in range(len(noisy)):
    if valids[i]:
        x8 = apply_8bit_wrap(int(noisy[i]))
        sym, valid = mlse.step(signal_in=x8, signal_in_valid=1, rstn=1)
        if valid:
            dec_syms.append(sym)
dec = GrayDecodePAM6()
py_post = dec.decode(dec_syms)

# Post-FEC Statistics
fec = IL_FEC_Checker(seed=seed_val, n=544, t=15, m=10, n_interleave=1)
for bit in py_post:
    fec.step(bit)
ber_pre = fec.total_bit_errors_pre / max(1, fec.total_bits)
ber_post = fec.total_bit_errors_post / max(1, fec.total_bits)
```

Figure E1: *Input PRBS to output FEC blocks in software.*